Singly Linked-List

→ Syntax of Linked-List -

A Linked List is implemented using structure and pointers.

• Syntax of structure of a Node;

```
struct node
{
    int data; // data field
    struct node *next; // Pointer to the next node
};
```

• Syntax of creating a New Node Dynamically; (Single Node)

```
struct node *nptr;
nptr = (struct node*) malloc(sizeof(struct node));
nptr->data = val;
nptr->next = NULL;
```

→ here, malloc(sizeof(struct node)) allocates memory for the node

→ Typecast to (struct node *) to match pointer type (because malloc returns a void pointer)

→ nptr→data = store a data (val)

→ nptr→next = NULL (because only one node is created.)

VAL	NULL
data	next

Akash

// C program to create a new node

```
#include<stdio.h>
#include<stdlib.h>      // required for malloc()
struct node
{
    int data;  // data field
    struct node *next;  // pointer to the next node
};

int main()
{

    int val;  // value of new node
    struct node *nptr;  // new node

    printf("enter a value of new node");
    scanf("%d", &val);
    // creating a new node
    nptr=(struct node*)malloc(sizeof(struct node));

    // assigning data and next pointer

    nptr->data=val;
    nptr->next = NULL;
    printf("value of new node=%d", nptr->data);
    return 0;
}
```

Akash

→ Syntax for Head pointer (start) ;

```
struct node *start=NULL;
```

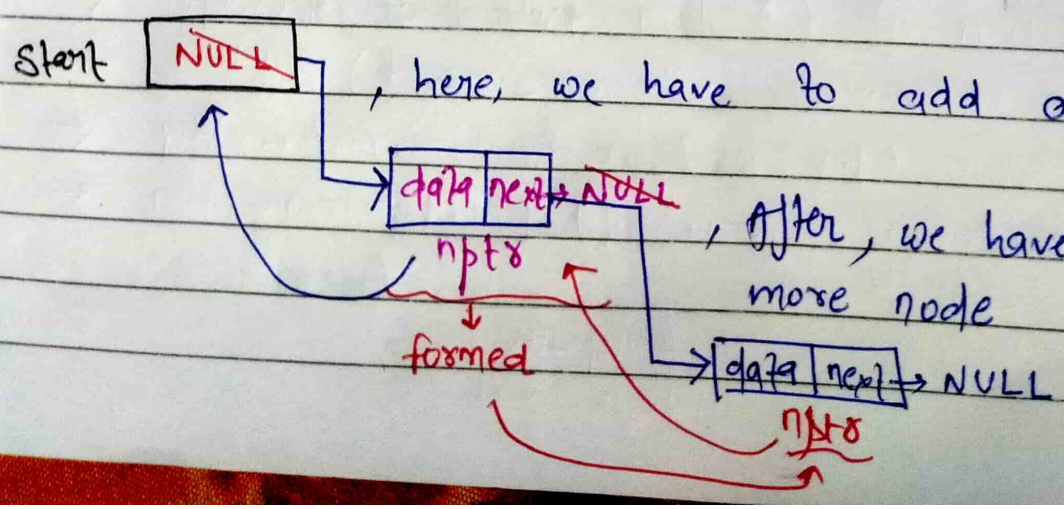
- struct node * → pointer to a node structure.
- start = NULL; → Indicates that the list is initially empty.

→ Syntax for adding nodes :-

```
struct node *create_List(struct node *start)// function
{                                // declaration to create a linked list
    // body
}
int main ()
{
    struct node *start = NULL; // header pointer(start)
    start = Create_List(start); // function name/call
    return 0;
}
```

↳ all list starts from begining i.e. start
or all link list is headed by start.

→ Algorithm to create a Linked - List.



Akash

step 1. Create nptr [new_node]

~~step 2. set start = NULL~~

step 2. set nptr → data = VAL;

step 3. set nptr → next = NULL;

step 4. IF VAL = -1
goto step 7

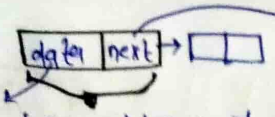
Step 5. IF (start == NULL)
5.1. start = nptr

Step 6. else
6.1. ptr = start
6.2. Repeat until ptr → next != NULL
{
ptr = ptr → next;
}
6.3. ptr → next = nptr

Step 7. Exit

Akash

Node →



→ ptr→next is the base address of next node

ptr→info

base address of this node is ptr

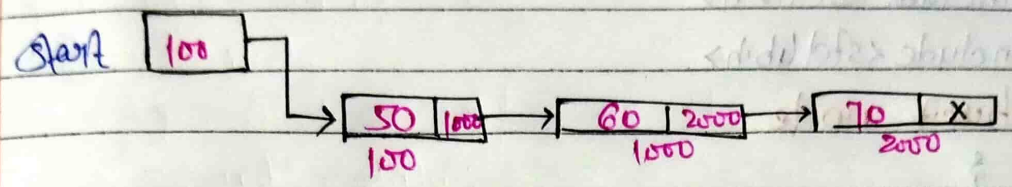
```
// C program to create a Linked List

#include<stdio.h>
#include<stdlib.h>
struct node
  {
    int data;
    struct node *next;
  };
  // function to create a Linked List
  struct node *create_list(struct node *start)
    {
      int val;
      struct node *ptr, *nptr;
      printf("enter -1 to end\n");
      scanf("%d",&val);
      while(val!=-1)
        {
          // creating a new node
          nptr=(struct node *) malloc(sizeof(struct node));
          // assigning data and next pointer
          nptr→data=val;
          nptr→next=NULL;

        if(start==NULL)
          {
            start=nptr;
          }
        else
          {
```

Akash

```
    ptr=start;

    while(ptr->next != NULL) // traversing
    {
        ptr=ptr->next;
    }
    ptr->next =nptr; // Link new node at end
  }

  printf("enter data");
  scanf("%d",&val);
 }
 return start;
}

int main()
{
  // header pointer(start) - initialize an empty list
  struct node *start=NULL;
  start = create_list(start); // function name/call
 return 0;
}
```

Akash

→ Algorithm to display a linked list;



Node is already created !!.
we have to display it i.e. to print.

Step 1.
Set ptr = start

Step 2. IF (ptr == NULL)
print "Empty list";
goto step 4.

Step 3. Repeat until (ptr != NULL)
{
print ptr → data;
ptr = ptr → next;
}

Step. 4. Exit.

Akash

// c program to Create and display Linked List.

```
#include <stdio.h>
#include <stdlib.h>
struct node
  {
     int data;
     struct node *next;
  };
struct node *create_List(struct node *start)
  {
     int val;
     struct node *ptr, *nptr;
     printf("enter -1 to end \n");
     scanf("%d", &val);
     while(val != -1)
       {
         nptr = (struct node*) malloc(sizeof(struct node));
         nptr->data = val;
         nptr->next = NULL;
           if(start == NULL)
             {
               start = nptr;
             }
         else
           {
             ptr = start;
             while(ptr->next != NULL)
               {
```

Akash

```
        ptr = ptr->next;
    }
        ptr->next = nptr;
    }
    printf("enter data");
    scanf("%d", &val);
    }
    return start;
}
// function to display a linked list
struct node *display_list (struct node *start)
    {
        struct node *ptr;
        ptr = start;
        if( ptr == NULL)
            {
                printf("empty list");
            }
        while (ptr != NULL)
            {
                printf("%d ->", ptr->data);
                ptr = ptr->next;
            }
        return start;
    }
int main()
    {
        struct node *start = NULL;
```

Akash

```
start = create_list(start);
start = display_list(start); // function name/call
return 0;
}
```

Akash